All artifacts maintained in to Github Markdown format and store diagrams draw.io – reference links to your TA

Submit each architectural and ER diagram to the project repository under docs/. For every diagram, include: (1) the editable .drawio source file, (2) an exported .svg rendering, and (3) a Markdown page explaining the diagram's purpose and assumptions. Commit source and export together. The repository version is the authoritative submission; external file-sharing links and image-only submissions will not receive full credit.

Phase 1: Planning & Design (Weeks 1-4)

Week 1: Business analysis & requirements gathering

- Group receives assigned business case
- Conduct stakeholder analysis
- Draft initial functional requirements document

Week 2: Functional requirements document finalization

- Complete sections:
  - User stories/use cases
  - Transaction flows
  - Security requirements

Week 3: Technical design document and ER diagram

- Create schema diagram with:
  - Transaction tables
  - Roles/entitlements
  - Audit logging structure
  - Denormalized reporting views
- Define microservice architecture

Week 4: System architecture approval

- Finalize:
  - API specifications

- Data flow diagrams
- wireframes

Phase 2: Development (Weeks 5-8)

Week 5: Database implementation

- Create MySQL schema
- Implement base tables with indexes/constraints
- Entitlements
- Database commit/rollback demo (see separate notes on this in Canvas)

Week 6: Mid-Project Review

- Present working schema
- Demo basic CRUD operations
- SQL Reports/transactions
- Submit progress report

Week 7: Microservices development

- Build extraction/update services
- Implement audit logging features

Week 8: Web interface prototype

- Create basic UI framework
- Connect to backend services

Phase 3: Testing & Deployment (Weeks 9-12)

Week 9: System integration

- Connect all components
- Implement security controls

Week 10: User acceptance testing

- Conduct security audits
- Performance testing

Week 11: Final adjustments

- Optimize reporting views
- Documentation completion

Week 12: Final Review

- Live system demo
- Code walkthrough
- Submit final deliverables

Evaluation Rubrics

Mid-Project Rubric (Week 6)

Criteria	Excellent	Proficient	Developing	Unsatisfactory
Requirements Documentation	Complete FRD with clear traceability	Minor gaps in use cases	Incomplete sections	Major omissions
Schema Design	Normalized tables + denormalized views with indexes	Basic structure implemented	Partial implementation	No working schema
Process Compliance	Regular commits + meeting all milestones	Minor delays in deliverables	Irregular progress	No tangible outputs
Presentation	Clear demo of working components	Partial functionality shown	Conceptual walkthrough	No demo

Final Project Rubric (Week 12)

Criteria	Weight	Exemplary	Competent	Needs Improvement
Functional Requirements	20%	All use cases implemented[1][2]	MVP achieved	Major gaps
Technical Implementation	30%	Optimized schema + secure services	Working system	Partial functionality
Reporting & Auditing	15%	Real-time audits + denormalized views	Basic logging	Missing components

Security	15%	Role-based access + encryption	Basic auth	No security measures
Usability	10%	Intuitive UI + error handling	Functional interface	Broken workflows
Documentation	10%	Complete system specs + deployment guide	Basic comments	No supporting docs

Group Management Recommendations

1. Use Git for version control with enforced branching strategy
2. Conduct weekly standups with progress reports
3. Implement peer evaluations using rubric-based assessments
4. Require individual contribution logs (minimum 4 commits/member)
5. Schedule TA check-ins every 2 weeks for guidance

Example showing github structure for draw.io submissions and .svg

```text
project-root/
├── docs/
│   ├── architecture/
│   │   ├── system-context.drawio
│   │   └── system-context.svg
│   └── database/
│       ├── erd.drawio
│       └── erd.svg
└── README.md
```

- `.drawio` is the required, editable source of truth.
- `.svg` is ideal for architectural and ER diagrams: sharp at any zoom, generally small, and renders in GitHub Markdown.
- PNG is acceptable if students cannot export SVG, but use a sufficiently large resolution.

Draw.io supports saving diagram files in GitHub, and diagrams can be edited directly in browser-based `github.dev` with a draw.io extension. GitHub storage also gives each commit a history of diagram changes. drawio +1